

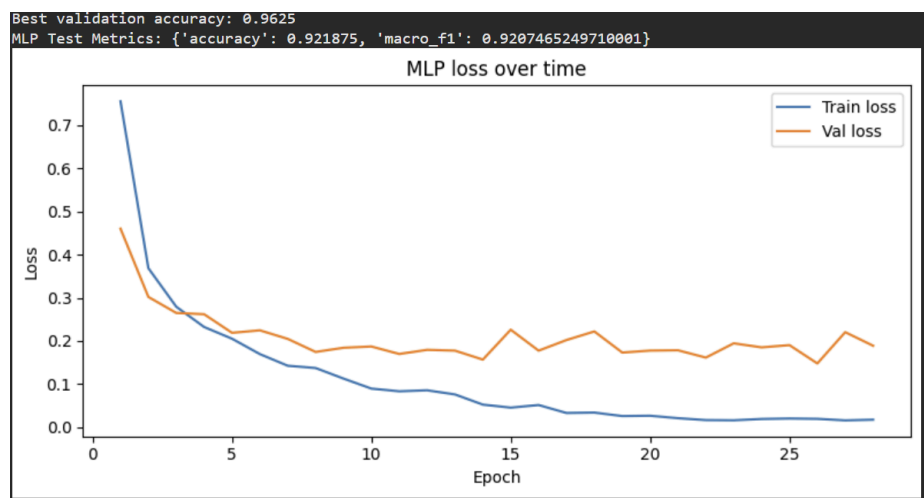
Section 1:

The problem I'm trying to solve is whether I can train a model to detect certain brain tumors on an MRI machine. I will do this by going over each image, and the model will either predict whether the image is a tumor and describe its type, or that it is not a brain tumor. The dataset contains 4 classes that contain 400 images, making the model have a fair and unbiased evaluation when being trained. The 4 classes are named glioma, pituitary, meningioma, and notumor. The dataset is also a multi-class classification problem.

The data also contains 512X512-pixel images and 7200 human brain MRI images categorized into four classes. This is an interesting problem because if this model is successfully able to predict brain tumors for my project, then it could also be used in real life to find brain tumors in patients, so that doctors don't have to manually look for them using the MRI images. So the medical world will benefit heavily if machine learning models can predict brain tumors very accurately. My dataset contains 5,600 training data images and 1,600 testing data images. I further divided the training set so that the training set contained 4,480 images and created a validation set with 1,120 images, while ensuring the data remained balanced. There are no class imbalances in the dataset since each class has an equal number of images being used. However, one limitation is that the dataset doesn't include the ages of the brains scanned or the gender, so we have less data on those metrics. Another limitation is that most of the images in the database are black and white MRI images and could fail on different MRI scans that are in color, so the model might be less accurate on those MRI images. Here is the link:

<https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset>.

Section 2:



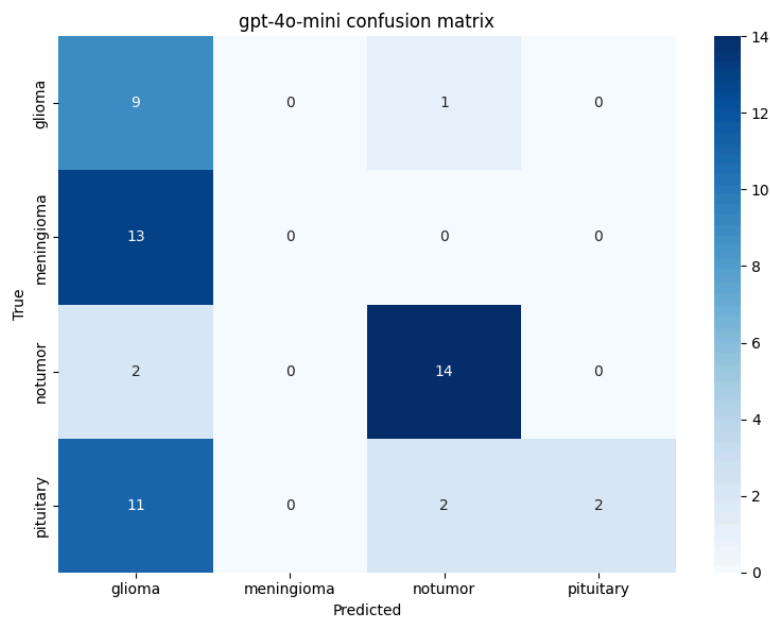
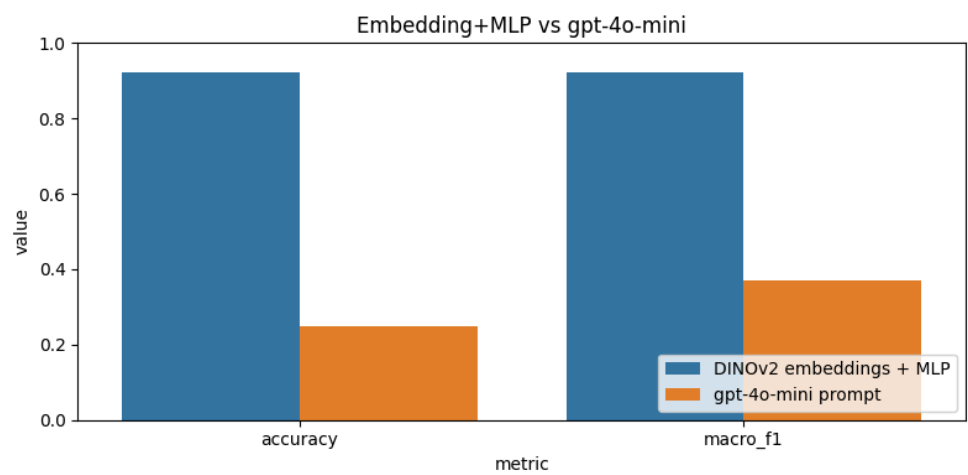
The pretrained encoder that I used was DINOv2-small, and the dimensional embedding was 384. My architecture had 2 hidden layers with a hidden dimensionality of 512 X 128. I also used RELU as an activation function because it is a nonlinear activation function, so it will be useful for my model in learning nonlinear complex patterns. I also used a dropout of 0.2 so that I could reduce any overfitting if there was any. However, I wanted to make the dropout very low because I didn't want it to set most of the values to 0 during training. The number of epochs I used was 28, the batch size I used was 256, the learning rate I used was 0.001, and the patience number I used was 6. I used the Adam optimizer for my architecture because I wanted to use the techniques of momentum and a pre-parameter learning rate, so that my model could adaptively change learning rates per parameter and reduce noise at the same time. The reason why I chose 28 for the number of epochs is that when I set my epoch to 10, my accuracy only went to 94%. However, when I increased the number of epochs, my accuracy kept increasing until I went up to 28 epochs, which is when I had a 96.25% for my best validation accuracy. I think this happened because when you use more epochs, the model has more time to train and get better, but if you use fewer epochs, then the model has less time to learn. However, when I tried increasing the number of epochs to a higher number, like 70, my accuracy wasn't able to go up any higher, likely because I set the patience to 6, which caused the accuracy to stop increasing and max out at this percentage. I also used a multi-layer MLP classifier because the database that I am using has multiple classes. Another thing I added to my architecture was to add transfer learning to freeze the DINOv2 foundation model weights, so that I could keep the MLP classification head updated. I used transfer learning so that my model could take the knowledge learned from one task and apply it to a new, related task.

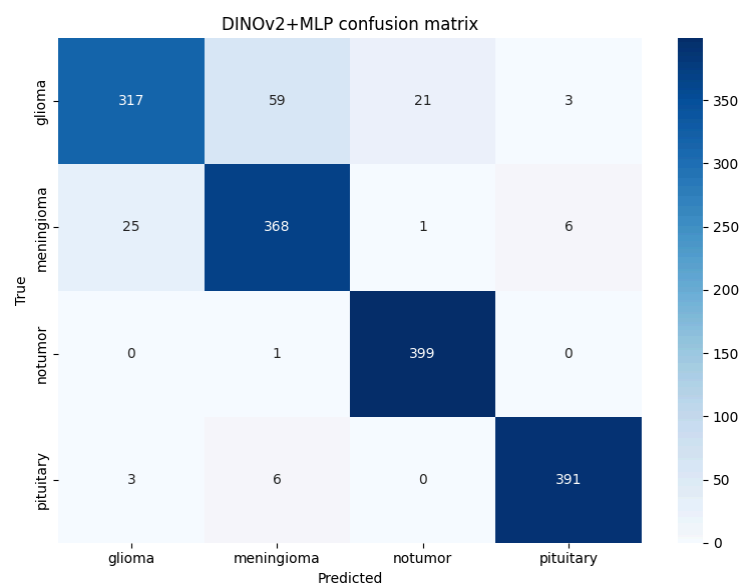
Section 3:

The LLM I chose was ChatGPT, and the model of ChatGPT that I used was gpt-4o-mini. I used ChatGPT's API because I thought it would be very good at classifying MRI images of brain tumors. Another reason why I used ChatGPT is that I know that the model is trained on vast amounts of rich data, so it should be able to classify the images pretty accurately. The exact prompt that I gave ChatGPT was: "I want you to classify Brain Tumor MRI dataset images. Can you please choose one label from the classes: {'', '.join(class_label)}'. Return only the label word". I tuned this prompt by making sure that the only labels ChatGPT can use are the class names, so that it doesn't classify an image that is outside of that window. This makes it easier for the outputs to be parsed and used. I also made my prompt very simple because I wanted to test GPT's reasoning capabilities. The prompting that I used was zero-shot prompting since I didn't provide examples in the prompt. I just asked the question since I thought that the task would be very simple for ChatGPT. When the prompt was received, I used Regex for prompting to extract the final class label from the LLM's text response. I did this through a function called `parse_label`, where I lowercased the response, then in the function I searched for any of the class

names in the dataset, like notumor or glioma. I searched for them using the word-boundary regex and returned one of the four class names as the prediction. However, if the response from the AI didn't contain one of the four class names, then it would be returned as unknown. The AI model was tested using a random subset of 100 images, and I used seeding to maintain the results of ChatGPT's classifications. Since I used a random subset, the AI model never predicted meningioma, likely because this type of brain tumor was not in the random subset of 100 images, or there were fewer images of this type of brain tumor compared to others. So the AI model never predicted this type of brain tumor, which is why the metrics for it are all 0.

Section 4:





MLP model results table:

	Precision	recall	F1
glioma	0.92	0.79	0.85
Menigioma	0.85	0.94	0.88
notumor	0.95	1.00	0.97
Pituitary	0.98	0.98	0.98
Accuracy			0.92
Macro-F1			0.92

gpt-4o-mini model results table:

	Precision	recall	F1
glioma	0.26	0.90	0.40
Menigioma	0	0	0
notumor	0.82	0.88	0.85
Pituitary	1.00	0.13	0.24

Accuracy			0.46
Macro-F1			0.37

In the MLP model, the failure case that was the most common was for identifying whether a tumor was a glioma tumor, because in the chart, the model either predicted that the tumor was a meningioma or that there was no tumor. We can see this in the misclassification examples because the MLP model would either predict no tumor, or it would predict the wrong type of tumor, like meningioma. The reason why my model must be failing on these cases is that the model is not understanding how certain tumor pixels are different from each other, or how certain pixel images represent a tumor. The model could be confusing the white pixels in the MRI with different parts of the brain, or the model could be getting confused when the MRI image is flipped upside down or in a different position, where it's harder to detect the tumor. This could then result in my model misclassifying certain tumors or classifying the scan with no tumors. This suggests that the model doesn't use that many flipping or rotating forms of data augmentation, so adding this could further improve its accuracy. In ChatGPT's model, the most common failure cases were misclassifying a pituitary tumor as either a glioma or predicting that it had no tumor. Another failure case was misclassifying meningioma as glioma.

ChatGPT could be misclassifying these images because the 100 samples it got were harder to determine, since in some of the images, the pixels of the tumors are more black and blend in a little bit more with the brain. This suggests that ChatGPT was never trained on MRI images that were harder to view. So ChatGPT starts to misclassify these images with no tumor or different types of incorrect tumors. In the results, we can see that ChatGPT never predicted meningioma, which is why the recall, precision, F1, and accuracy are all 0 for that class of tumor. My reasoning as to why this happened is that there weren't as many MRI images for meningioma in the dataset, and also because the images used for meningioma could have been harder to classify, resulting in ChatGPT being unable to ever predict meningioma. This suggests that ChatGPT may not have a lot of image data on meningioma in its AI. In the misclassified images, we can see that for one of the misclassifications, glioma is predicted, but the actual label is meningioma. However, in the MRI image, it is hard to notice where the tumor is, so ChatGPT misclassifies it as a glioma tumor. I also think that adding more data augmentation to ChatGPT's AI model could improve its accuracy, because it did misclassify on images where the brain was scanned in a different rotation or flipped from the side.

The metrics I chose were accuracy, precision, recall, and F1 score. I chose the accuracy metric because I knew that the dataset was balanced, so accuracy would be less misleading. However, I still chose to use precision and recall, so I could measure if the model had high false negatives or high false positives, depending on how low the precision and recall were. This is so that if the accuracy of my model is low, I can determine why based on whether the recall or precision is

low, since I will know if the model's predictions are either high false positives or negatives. I added the F1 score as one of my metrics because I wanted to compare it with my accuracy to see if they were relatively the same. This way, I know for sure that the dataset is balanced. Another reason why I added the F1 score as part of my metrics is that, for the LLM prompting, I only used a random subset of 100 samples to test ChatGPT's classifications. Since the samples are random, this could result in the data being imbalanced, so I added the F1 score to see if the accuracy could be misleading. The best validation accuracy for the MLP model was 96.25%, and its average macro-F1 was 92%. while the best accuracy for GPT-4o-mini was 46.3%, and its average macro-F1 was 37%. The reason why the MLP model did better than ChatGPT is that the MLP model was trained on the data, but the ChatGPT model was only tested on the images, so it performed lower than the MLP model. Another reason why MLP did better is because of its use of embeddings, which helped capture semantic meaning in the data.

Google Slides:

https://docs.google.com/presentation/d/151wi-bmL9Ufcrteje5t0p7bNbfoOI3P_hmoQG3jChNM/edit?usp=sharing